IntelliDigest

A multi-source, per-user RAG research assistant

LangChain • ChromaDB • Groq (Llama 3.3) • FastAPI

IntelliDigest ingests documents and live news into a private, per-user vector knowledge base, then answers questions over it with grounded, source-cited responses. A second, isolated knowledge base powers a tool-calling customer-support agent with its own ticketing system. Everything runs behind a single FastAPI service with pluggable LLM providers and an offline fallback.

Author
Ramez Asaad

Type
Full-stack AI application

Document
Architecture & design

Revision
July 2026

At a glance

Retrieval	Per-user Chroma collections, MiniLM embeddings, semantic search
Generation	Persona-aware RAG with rolling conversation memory
Agent	LangChain `AgentExecutor` + tools for the support surface
Resilience	Groq primary → Ollama fallback via `RunnableWithFallbacks`
Isolation	Two Chroma collections keep private uploads out of support answers
Surface	FastAPI REST API + hand-built HTML/CSS/JS single-page frontend

1 Overview

IntelliDigest is a research assistant built on the **retrieval-augmented generation** (RAG) pattern. A signed-in user uploads documents (PDF, DOCX, Excel, TXT) or pulls in live news articles; the system parses, chunks, embeds, and stores them in a vector database scoped to that user. When the user asks a question, the relevant chunks are retrieved and handed to a large language model together with a selected persona and a compressed history of the conversation. The answer comes back grounded in the user's own material, with citations.

A second, deliberately separate surface handles **customer support**. It is a tool-calling agent that searches a curated support knowledge base, classifies the issue, and can open a ticket in a SQLite store — but it never touches the user's private documents. This split is the central design decision of the project and is described in Section 3.

1.1 What the system does

- **Ingests** multiple formats and live news into a per-user vector store.
- **Answers** questions over that store with grounded, source-cited responses.
- **Adapts tone** through five personas (Technical, Business, Academic, Casual, Political).
- **Summarizes** content with both a single-pass (*stuff*) and a *map-reduce* chain.
- **Remembers** multi-turn context by compressing older exchanges into a rolling summary.
- **Supports** users through an isolated tool-calling agent with SQLite ticketing.
- **Authenticates** with email/password (JWT), optional Google OAuth, and SMTP password reset.
- **Degrades gracefully** to a local Ollama model when the primary provider errors or rate-limits.

1.2 Design principles

- **Isolation by default.** Each user's data lives in its own vector collection; support content is a separate collection again. Nothing leaks across boundaries.
- **Grounded, not guessed.** The research surface always retrieves before it generates and cites what it used.
- **Never dies in a demo.** Provider failures fall through to a local model rather than surfacing an error.
- **One process, no build step.** A single FastAPI app serves the API and a dependency-free frontend.

2 System architecture

IntelliDigest is a single FastAPI process (`server.py`) that serves a static frontend and initializes its LangChain components lazily on startup, off the event loop, so the HTTP server binds immediately and health checks pass while the embedding model loads. One process holds the embedding model, both Chroma collections, and the SQLite databases; external calls go out to the LLM provider, NewsAPI, and OAuth/SMTP.

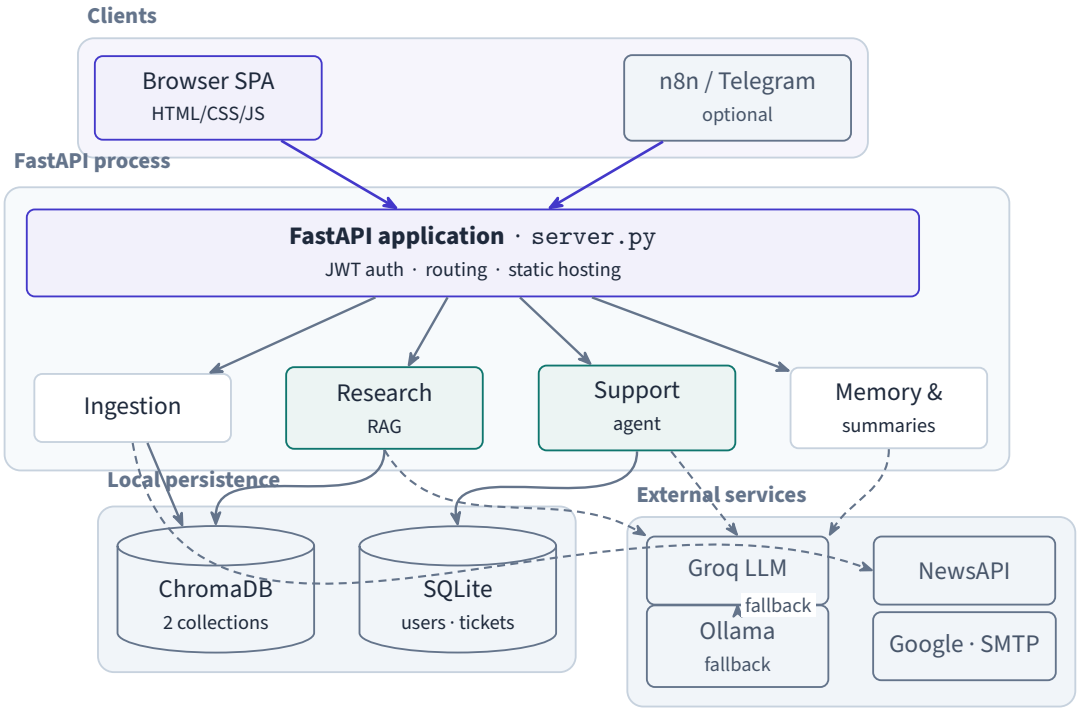

Figure 1: High-level runtime. Clients reach a single FastAPI application that fans out to four internal capabilities backed by local persistence and external APIs.

Startup contract. If `GROQ_API_KEY` is present, the app builds the vector engine, bootstraps the support knowledge base, and wires up the research flow. If it is absent, the components stay uninitialized and the routes that need them return 503 rather than crashing — the server still starts and serves the frontend.

3 The dual knowledge-base design

Embeddings come from HuggingFace `all-MiniLM-L6-v2` (384-dimension, normalized), shared by both stores. Chroma then keeps two kinds of collection that never mix:

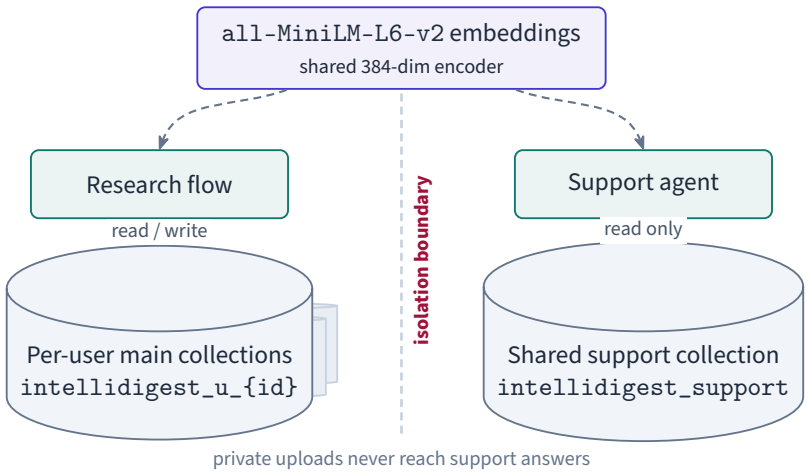

Figure 2: Retrieval isolation. The research flow only ever reads a user's own collection; the support agent only ever reads the shared, curated support collection.

- `intellidigest_u_{id}` — one **main** collection per authenticated user, holding their uploads, ingested news, and any n8n-pushed content. Named from a stable hash of the user id.

- `intellidigest_support` — a single **shared** collection seeded from curated `support/kb/*.md` files, bootstrapped on first run if empty.

The payoff is a hard guarantee: a user's private documents can never surface inside a scripted support answer, and one user's uploads are never visible to another. The isolation is structural, enforced by which collection each code path is allowed to open — not by a filter that could be forgotten.

4 Data flows

4.1 Ingestion into the main knowledge base

Everything that adds text to a user's collection follows the same path: *raw content* → *chunk* → *embed* → *Chroma*. Uploads are parsed per format and semantically chunked on sentence boundaries; news articles are fetched from NewsAPI and de-duplicated by URL before embedding.

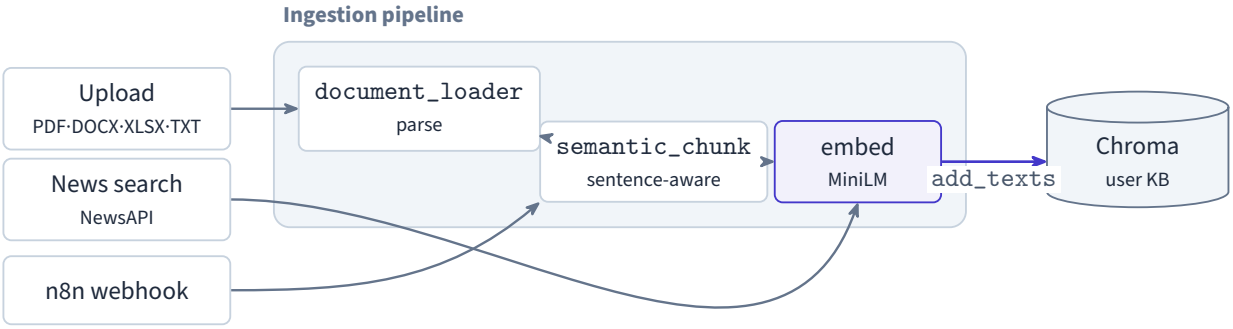

Figure 3: Three sources converge on one embedding-and-store path, all scoped to the authenticated user's collection.

4.2 Research chat

The research surface retrieves the top-k chunks from the user's collection, builds a prompt from the persona, the retrieved context, and the compressed chat history, then calls the LLM stack. The answer is parsed for inline follow-up suggestions and returned with its sources; the exchange is folded back into memory.

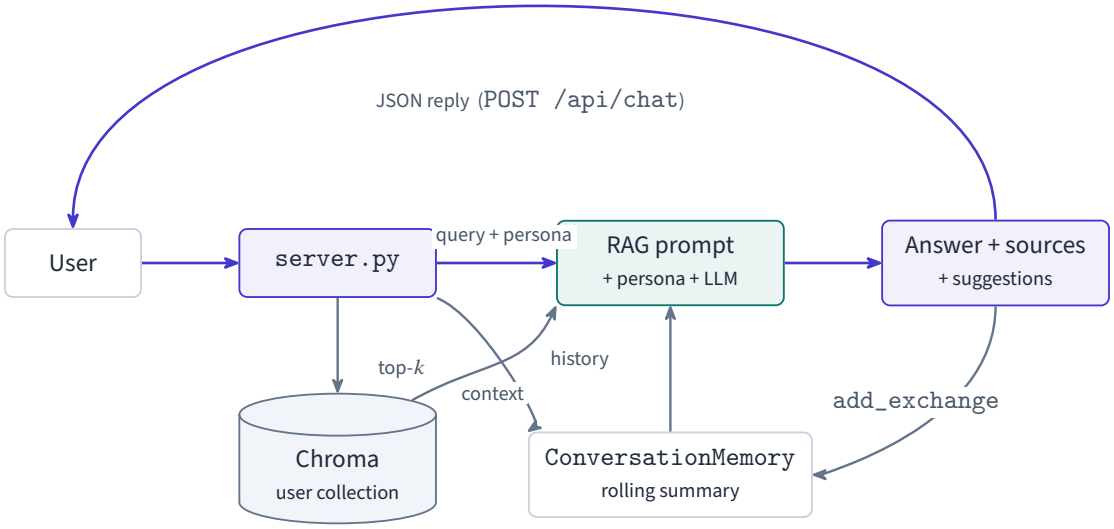

Figure 4: Request path for `POST /api/chat`. Retrieval and memory both feed the single generation call; the reply updates memory before it returns.

Memory compression. `ConversationMemory` keeps recent turns verbatim and, once history grows past a threshold, compresses older exchanges into a running summary with the LLM. Long conversations stay inside the context window without losing their thread.

4.3 Support agent and ticketing

The support surface is a genuine LangChain `AgentExecutor` built with `create_tool_calling_agent`. The model decides which tools to call:

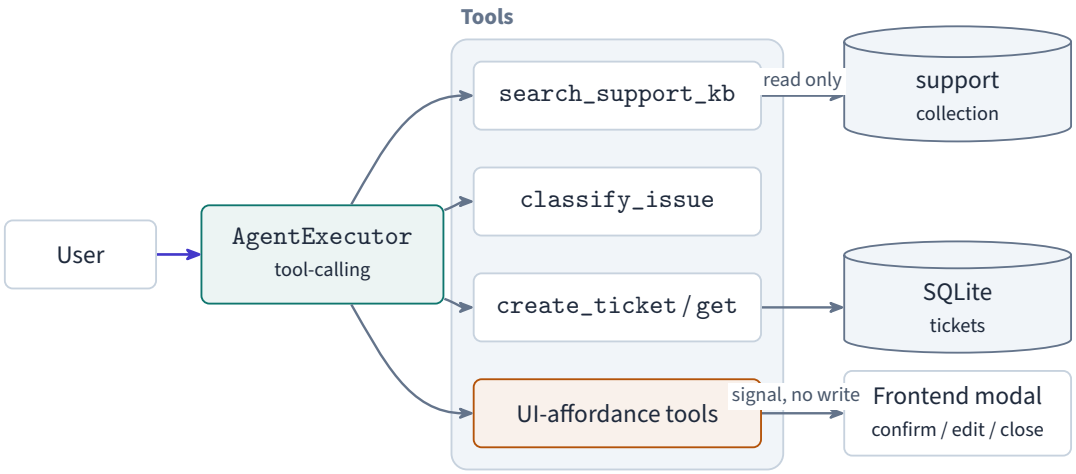

Figure 5: The support agent reasons over four tool families. Knowledge search is restricted to the support collection; UI-affordance tools only signal the frontend, they never mutate the database.

- `search_support_knowledge_base` — retrieves from the support collection *only*.
- `classify_issue` — assigns a category and priority.
- `create_ticket` / `get_ticket` — read and write SQLite ticket rows.
- **UI-affordance tools** — `show_confirm_close`, `show_edit_ticket`, and friends emit a signal the frontend turns into a confirmation modal. They perform *no* database writes.

Destructive changes (closing or editing a ticket) therefore go through an explicit REST call after the user confirms in the UI — the model can propose a mutation, but it cannot silently commit one.

5 LLM strategy and resilience

A single factory (`chains/llm_factory.py`) builds every chat model in the system. It supports a bring-your-own-key choice of provider (Groq, OpenAI, or Gemini) and always wraps the primary model with a local Ollama fallback via LangChain's `RunnableWithFallbacks`. Rate limits, timeouts, connection errors, and server errors are all caught and retried against the local model, so a provider outage degrades quality instead of breaking the request.

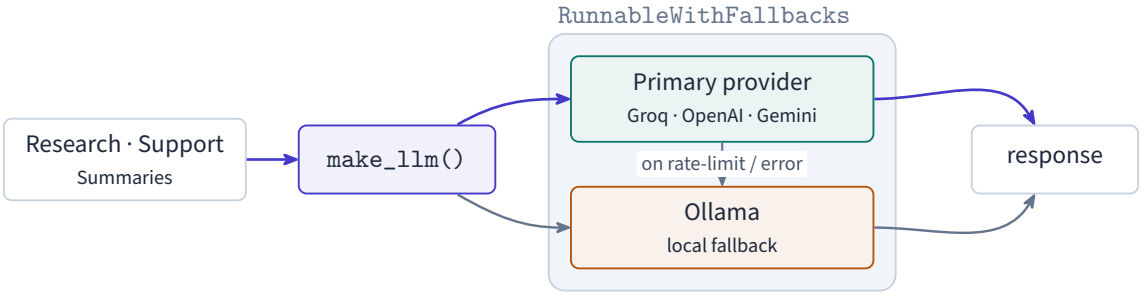

Figure 6: Every chain and agent in the system is constructed through one factory, so the fallback behavior is uniform across research, support, and summarization.

6 Interface surface

The frontend is a single hand-written HTML/CSS/JS page — no framework, no build step — talking to a REST API. The table below groups the most important endpoints; every data route requires a JWT bearer token.

Area	Endpoint	Purpose
Auth	`POST /api/auth/register`	Email/password sign-up, returns JWT
Auth	`POST /api/auth/login`	Login, returns JWT
Auth	`GET /api/auth/google`	Begin Google OAuth
Ingest	`POST /api/upload`	Parse, chunk, and embed a document
Ingest	`POST /api/news/search`	Fetch and ingest news articles
Research	`POST /api/chat`	Grounded, persona-aware answer + sources
Research	`GET /api/search`	Semantic search over the main collection
Research	`POST /api/summarize`	Brief (stuff) or detailed (map-reduce) summary
Support	`POST /api/support/chat`	Tool-calling support agent
Support	`GET /api/tickets`	List the user's tickets
Support	`POST /api/tickets/{id}/close`	Close a ticket after confirmation
Ops	`GET /health`	Liveness probe (does not block on model load)

Table 1: Representative REST surface. Per-user isolation is enforced server-side from the authenticated identity in the token.

7 Technology stack

Layer	Choice and rationale
Orchestration	**LangChain** — LCEL chains, `AgentExecutor`, prompt templates, memory
Primary LLM	**Groq** (Llama 3.3 70B) — fast inference for chat, RAG, and summaries
Fallback LLM	**Ollama** (local) — keeps the app answering when the provider fails
Embeddings	**HuggingFace** `all-MiniLM-L6-v2` — compact, CPU-friendly, normalized
Vector store	**ChromaDB** — persistent, per-user main + shared support collections
API	**FastAPI** — async routes, dependency-injected auth, static hosting
Persistence	**SQLite** — users and support tickets under a configurable data root
Frontend	**Vanilla HTML/CSS/JS** — a design-tokened SPA with no build step
Automation	**n8n** (optional) — webhook bridge for Telegram delivery

Table 2: Deliberately lean stack: one process, local persistence, and a single orchestration library across every AI surface.

7.1 Engineering notes

- **Deferred heavy imports.** Embeddings and LangChain load in a background thread so uvicorn binds before the model is ready — important for container health checks.
- **Per-user collection naming.** A user id is normalized to a stable 32-character suffix (hashed when needed) so a collection maps deterministically to an account.
- **News de-duplication.** Articles are checked against existing URLs in the collection before embedding, preventing duplicate chunks on repeated searches.
- **Confirmation-gated mutations.** The support agent surfaces intent to the UI; the actual write happens over REST only after the user confirms.
- **Tested auth core.** The authentication and token paths are covered by an automated `pytest` suite.

Summary. One FastAPI process, two isolated vector collections, one LLM factory with a built-in safety net, and a clean split between a grounded research surface and a tool-driven support surface. The architecture optimizes for privacy, resilience, and a demo that keeps working when the network does not.